A Full-system, Programmable, and Extensible In-Memory Computing Simulation Framework for Deep Learning

Kaining Zhou, Jian Huang, Nam Sung Kim, Naresh Shanbhag
University of Illinois at Urbana-Champaign, IL, USA
{kainingz, jianh, nskim, shanbhag}@illinois.edu

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132463

***Abstract*—In-memory computing (IMC) has established itself as an attractive alternative to hardware accelerators in addressing the memory wall problem for artificial intelligence (AI) workloads. However, designing programmable IMC-based computing platforms for today's large generative AI models, such as large language models (LLMs) and diffusion transformers (DiTs), is hindered by the absence of a simulator that is able to address the associated scalability challenges while simultaneously incorporating device and circuit-level behaviors intrinsic to IMCs. To address this challenge, we present *IMCsim*, a versatile full-system IMC simulation framework. IMCsim integrates software runtime libraries for AI models, introduces a new set of ISA extensions to express common tensor operators, and provides flexibility in mapping these operators to various IMC architectures. As such, IMCsim enables designers to explore trade-offs between performance, energy, area, and computational accuracy for various IMC design choices. To demonstrate the functionality, efficiency, and versatility of IMCsim, we model three types of IMCs: (1) embedded non-volatile memory (eNVM)-based, (2) SRAM-based, and (3) digital IMCs. We validate IMCsim using measured data from two laboratory-tested IMC prototype ICs—a $22\,\mathrm{nm}$ MRAM-based IMC and a $28\,\mathrm{nm}$ SRAM-based IMC—and a digital IMC design in $28\,\mathrm{nm}$. Next, we demonstrate the utility of IMCsim by exploring the architectural design space to obtain insights for maximizing utilization of IMC-based processors for diverse workloads—ResNet-18, Llama, and a DiT—using the three IMC types. Finally, we employ IMCsim as a design tool to obtain an efficient chip architecture and layout in $28\,\mathrm{nm}$ for a lightweight DiT.**

I. Introduction

In-memory computing (IMC) [17] is a promising approach that incorporates computation into memory arrays, thereby alleviating the memory bandwidth bottleneck of conventional von Neumann architectures. By enabling data-intensive tasks such as matrix-vector multiplication (MVM) within memory, IMC is well-suited for AI workloads built upon deep nets [12] and transformers [28], exemplified by a collection of IC prototypes [6], [7], [11], [15], [16], [26], [27], [29], [31].

The design of an IMC-based system is complex. Designers innovate across the compute stack: device, circuit, architecture, and workload mapping. Therefore, there is a critical need for a fast IMC simulator to enable the design and validation of full IMC-based systems. Such a simulator should provide full-system, cycle-level run-time behavior, with support for instruction set architecture (ISA), while capturing the impact of device and circuit non-idealities.

Previously developed IMC simulators (see Table I) fall into one or more of the following three classes: 1) **resistive crossbar macro simulators**: these characterize circuit-level metrics of IMC crossbar macros [14], [19], [33]; 2) **architecture behavioral simulators**: these simulate behavior of IMC architectures when executing applications [2], [32]; and 3) **design space exploration tools**: these enable fast exploration of efficient IMC designs using statistics-based modeling and predefined mapping [1], [33]. In most cases, these existing simulators can handle only small-scale workloads without considering the impact on an end-to-end IMC system, and therefore are unable to serve as a design tool for an actual chip implementation.

To this end, we develop IMCsim, a versatile full-system IMC simulation framework for designing IMC-based systems capable of executing large AI models such as large language models (LLMs). Unlike standalone static device/circuit characterization tools, IMCsim integrates off-chip memory access and run-time environment into the simulator. To support simulation of various types of workloads on diverse architectures, IMCsim provides flexible programmable interfaces (see Section II-B). Further, IMCsim allows designers to decompose deep learning (DL) kernels into basic tensor operators and proposes a set of extensible ISA that translates these operators into the instructions that can be executed on IMC processors. Furthermore, instead of focusing on a single type of memory device, IMCsim can support different memory technologies such as SRAM, ReRAM, and MRAM.

IMCsim enables cycle-level behavioral simulation by providing accurate timing for each circuit operation. It also employs well-acknowledged power models for various memory devices and IMC processors, for precise power estimation. To guarantee IMC accuracy, IMCsim develops a signal-to-noise ratio (SNR) model that quantifies the computational accuracy.

We implement IMCsim and integrate it into a full-system emulator, QEMU [5]. To evaluate the efficacy of IMCsim, we use three types of DL applications, including ResNet-18 [12], Llama [25], and Diffusion Transformer (DiT) [21] on SRAM-based and MRAM-based analog IMC (AIMC), as well as SRAM-based digital IMC (DIMC). We show the end-to-end performance, energy consumption, and computation accuracy from the simulation, validated by fabricated IC and complete layouts. As IMCsim reports detailed run-time information on the status, utilization, and energy of each component, it will benefit future IMC development by offering useful architectural insights. Overall, the contributions of this work include:

1) **Full-system, programmable, and extensible simulator**: we present IMCsim, a versatile simulator allowing researchers to utilize hardware libraries to model diverse IMC processors with ISA support atop various IMC devices. IMCsim is calibrated with data from fabricated chips and chip layouts. Its built-in expressive application programming interfaces facilitate designers to program and map workloads correspondingly. IMCsim is open-sourced at https://anonymous.4open.science/r/IMCsim-2B45/.
2) **Accurate and insightful evaluation for large-scale AI models**: we employ IMCsim's cycle-level run-time information, noise analysis, and interaction with external memory systems, to identify bottlenecks and highlight optimization opportunities when mapping Llama and DiT to three different IMC architectures, respectively.
3) **Design exploration targeting chip implementation**: we demonstrate the use of IMCsim to conduct a design space exploration for implementing an IMC chip layout for a lightweight DiT in $28\,\mathrm{nm}$ process technology. This exploration involves architectural and functional design, evaluation, verification, and optimization of the IMC system illustrating IMCsim's utility as a design tool.

We organize the rest of the paper as follows: Section II highlights

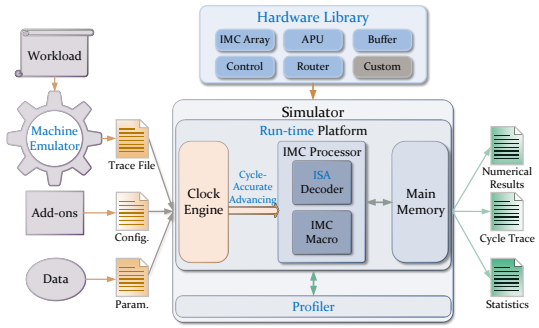


Fig. 1: System architecture of IMCsim.

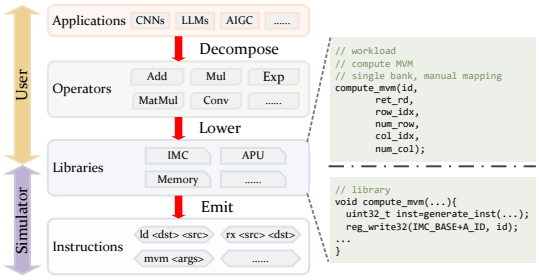


Fig. 2: Software stack and programming interface of IMCsim.

design features of IMCsim, Section III shows how validation is performed, and Section IV demonstrates multiple use cases to show IMCsim's efficacy, and Section V concludes the paper.

II. IMCsim Features

This section describes key IMCsim features including circuit and architecture model, cycle-level behavior, extensible ISA, run-time profiling, and others.

A. Hardware Library

IMCsim provides a hardware library comprising several "building blocks" (see Fig. 1) to enable designers to compose a diversity of IMC-based architectures.

Basic blocks: IMCsim has five common blocks found in current day IMC processors: an IMC array, an auxiliary processing unit (APU), a buffer, a controller, and a router. The IMC array is the core memory array for MVM computation with the following parameters: the number of rows and columns, the number of ADCs and adder trees, the I/O width, and their timing information. The APU is a digital block comprising several types of computational units, including fixed-point and floating-point ALUs. It can be organized and executed in a vectorized manner. The combined effort of the IMC array and the APU makes IMC capable of processing general AI workloads. The controller executes instruction fetch, and decode, for the IMC bank. The buffer is an on-chip SRAM that stores intermediate results and buffers data transmitted through the I/O. The router connects multiple IMC banks.

Custom blocks: Designers have the flexibility to incorporate their own components to this simulator, *e.g.*, sparsity detection units, and quantization units. Section II-F provides more details.

B. Programmability

Compared with related IMC modeling tools [14], [19], IMCsim provides programmability across diverse applications, architectures and devices (see Fig. 2). Users select an application and express it using common operators *e.g.*, those found in ONNX [3]. Next,

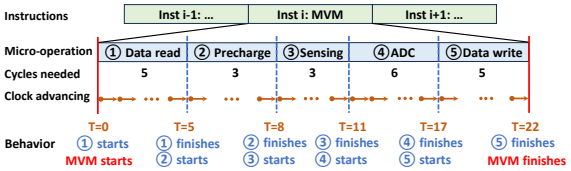


Fig. 3: Behavioral cycle-level modeling for MVM instruction.

software operators are mapped onto hardware-supported operations via library calls such as `compute_mvm`. Such flexibility allows users to run various applications and evaluate different mappings of applications on different architectures. Thus, IMCsim is not confined to a predefined set of workloads or architectures, making it highly versatile.

C. ISA Support

Compared to related tools [14], [32], IMCsim provides an additional level of flexibility via a specialized and extensible RISC style ISA to designers as they compose their IMC processor. Table II lists the most frequently used instructions obtained by analyzing DL workloads and IMC designs. Instructions in Fig. 2 adhere exactly to this ISA. The ISA allows for easy implementation and easily extensible to a specific accelerator. The trace file (Fig. 1) is formatted to drive IMCsim.

D. Cycle-level Simulation

IMCsim overcomes the limitations of existing tools in simulating complex workloads at system and application levels by offering cycle-level behavioral simulation, and balancing the abstraction between circuit and application while specifying operation timing. Applications are decomposed into detailed operators by the user, and translated into hardware instructions (last level in Fig. 2). Meanwhile, lower-level circuits are abstracted into building blocks (IMC arrays, APUs, buffers) ensuring a comprehensive simulation for system and application-level designs. IMCsim models timing by decomposing one instruction into several micro-operations, each of which is associated with a hardware component with an execution time. This association is determined by the user.

For instance, to process one MVM, the system needs to execute 5 micro-operations which the user derives from real circuit behavior and provides to the simulator. As shown in Fig. 3, at T=0, IMCsim begins to execute the micro-operation ① which is data read. The clock engine increments the time T until T=5, which is the cycle count for ①, denoting its completion. At T=5, the input vector for MVM is updated thereby simulating data read from on-chip memory. The simulation proceeds to simulate the precharge micro-operation ② next. Only after the data write micro-operation ⑤ is executed, is the MVM instruction completed and committed.

E. Run-time Support

IMCsim provides run-time support since it is necessary to expose errors to the user during execution. It also enables the user to identify bottlenecks and optimization opportunities unavailable at compile-time. We list IMCsim's run-time support as follows: (1) determine run-time behavior per cycle per component, *e.g.*, the specific bank executing a specific instruction and buffer addresses being accessed; (2) perform numerical computations to monitor intermediate data easily; (3) monitor component behavior and extract metrics such as energy and utilization, *i.e.*, it dynamically profiles metrics instead of predicting them statically at compile-time.

TABLE I: Landscape of various IMC simulators

Work	Feature	Type of memory	Application capability	Cycle-level	Run-time support	ISA support	Noise analysis	Full-system simulation
PUMAsim [2]	PUMA-specific architecture simulator	Memristor	ML, CNNs	Yes	Yes	Yes	No	No
RxNN [14]	Fast crossbar simulation with non-ideality	ReRAM	CNNs	No	No	No	Yes	No
MNSIM [33]	CNN-specific IMC design framework	Memristor	CNNs	No	No	No	Yes	No
NeuroSim [19]	Device-accurate crossbar simulator	NVM	MLPs, CNNs	No	No	No	Yes	No
PIMulator-NN [32]	Event-driven crossbar simulator	NVM, SRAM	MLPs, CNNs	No	Unknown	No	No	No
CiMLoop [1]	Fast and flexible IMC modeling tool	User-defined	CNNs, Transformers	No	No	No	No	Yes
IMCsim (our work)	**Full IMC system design tool**	User-defined	General AI workloads	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

TABLE II: A snapshot of base ISA

31 25	24 20	19 15	14 12	11 7	6 0	
000000	X	rs1	000	rd	0000000	LOAD
000001	rs2	rs1	000	X	0000000	STORE
000010	rs2	rs1	000	rd	0000000	TX
000000	X	X	000	rd	0000001	MVM
000010	rs2	rs1	000	rd	0000001	ADD
000100	rs2	rs1	001	rd	0000001	FADD

F. Extensiblity

IMCsim preserves extensibility in two respects:

ISA extension: IMCsim's set of basic instructions (Table II) can be easily extended. For example, the ISA can be extended to support BF16 (16-bit brain floating-point) data arithmetic by: (1) specifying the format of the new instruction; (2) adding the corresponding functions in the software library; and (3) modify the tracing function in the machine emulator (Fig. 1) to correctly detect the execution of the instruction for collecting the traces. The ISA extension needs to be accompanied by an extension to the hardware as discussed below.

Hardware extension: IMCsim enables the user to include custom blocks to extend their hardware by: (1) creating a class, *e.g.*, class of BF16 hardware under the class of IMC macro, and elaborate its definition and function in the simulator; (2) profile the hardware in advance to characterize its metrics such as latency, energy, and area; and (3) define its behavior in each cycle as the clock is advanced; (4) add a function to advance its clock. Thus, the function of BF16 hardware will be invoked whenever the new BF16 instruction is executed.

G. SNR Estimation

IMCsim constructs an SNR model to address fixed-point digital-based and representative analog-based IMCs. Compute SNR is defined by the ratio of variance of the ideal output and variance of the deviation from the ideal (noise) output. Specifically, the compute SNR for a dot product (DP) of $\mathbf{w}$ and $\mathbf{x}$, ideal output y_o, the actual output y, noise η, then compute SNR is given as follows.

$$y_o = \mathbf{w}^{\mathbf{T}}\mathbf{x} = \sum_{j=1}^{N} w_j x_j \tag{1}$$

$$y = y_o + \eta \tag{2}$$

$$\text{Compute SNR} = 10\log_{10}\left(\frac{\sigma^2[y_o]}{\sigma^2[\eta]}\right)(\text{dB}) \tag{3}$$

where y_o is the ideal output, y is the actual output, and η is the deviation or noise. IMCsim takes the configuration and actual data as input, including the dimension of IMC arrays, bitwidth of ADCs (or output precision), inputs and weights, their bitwidth, and other parameters to compute y and then employs (3) to estimate the compute SNR. For AIMCs, the impact of circuit parameters such as thermal noise, deviation of capacitance, and supply voltage are incorporated through pre-characterization of the hardware components.

H. Full-system Integration

Main memory: IMCsim accounts for traffic to/from main memory in its simulation set-up. In the process, it reports the latency and energy of such off-chip memory accesses which can dominate the system latency and energy consumption. In the process, IMCsim measures the impact of accessing main memory by an IMC processor thereby providing a more complete assessment of the system to the user.

Full-system emulator: IMCsim is integrated into the full-blown generic machine emulator QEMU (Fig. 1) to further enhance its programmability and completeness as a design tool. IMC macros can be added to a machine as a peripheral device through MMIO or a PCI device. The user can define IMC macro specifications and program it in C to process DL algorithms by applying particular mapping schemes. After invoking QEMU, the transaction between the CPU and IMC device is captured automatically after which IMCsim begins its simulation.

III. Simulator Validation

We use three chips for validation: (1) measured data from a $22\,\text{nm}$ MRAM-based AIMC chip to calibrate one exemplary NVM-based IMC device; (2) measured data from a $28\,\text{nm}$ SRAM-based AIMC chip to be our representative volatile memory-based IMC device; (3) and simulated data from a $28\,\text{nm}$ DIMC chip.

Timing validation: IMCsim supports in-order, single-stage execution in the IMC processor for simplicity and ease of verification. We profile the hardware components to obtain their execution times for each micro-operation and provide it as inputs to IMCsim so that it can provide cycle-level timing behavior for the system. For example, for timing characterization of IMC banks, we obtain the latency of each micro-operation, such as data read/write from circuit simulations or chip measurements, to obtain the cycle-level timing for an MVM instruction as shown in Fig. 3. For digital components such as controller, fixed- and floating-point digital blocks, we simulate their behavioral RTL models for functional verification and synthesize them to obtain their latency. Lastly, we simulate the IMC system in IMCsim and compare the resulting system-level outputs with those obtained from software (Pytorch and C) as behavioral ground truth. This validates both the behavior and timing of the IMC system.

IMCsim allows for new designs and instruction extensions to be incorporated. To do this, the user can profile the unit cycle needed by new components in advance and include it to a cycle list maintained internal to the simulator.

Metric validation: For metrics besides latency, we characterize hardware in advance at the circuit level as well. We measure our IMC chips to obtain energy consumption for each stage and area overhead breakdown for the IMC array. In addition, we generate layouts and conduct post-extraction simulations to obtain the energy and area numbers. Further flexibility and extensibility are provided by add-ons in IMCsim, including [4] for SRAM and DRAM metrics, and [19] for SRAM and NVM metrics. Moreover, a technology scaling tool [23] is used, enabling users to explore the impact of technology scaling.

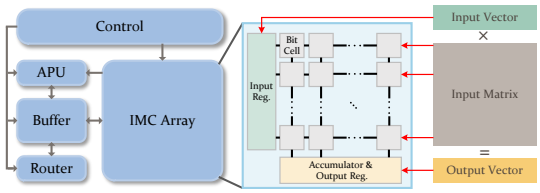


Fig. 4: IMCsim's flexible architectural template for composing IMC systems.

TABLE III: Design configurations employed in the case studies.

Component	Specification	Latency	Energy (fJ)	Area (mm^2)
DRAM	128b IO	25.8 ns/access	5.35×10^6	-
Buffer	192KiB, 128b IO	1.6 ns/access	29×10^3	1.00
Controllor	-	-	6×10^3/cycle	2.036×10^{-3}
Router	16b IO	16 B/2cycles	11	8.835×10^{-3}
APU	2 copies of ALUs	1-30 cycles	116-1477	0.19

IMC Array	Operation	Cycles	Energy (fJ)	Area (mm^2)
SRAM AIMC 512×512	Row read	4	1.02×10^5	0.19
	Row write	4	2.30×10^5	
	1b input driver	1	1.5	
	8b ADC	16	5.2×10^3	
MRAM AIMC 512×512	Row read	4	1.20×10^5	0.86
	Row write	50	4.6×10^7	
	1b sensing	2	98	
	8b ADC	8	3.1×10^3	
SRAM DIMC 512×512	Row write	4	2.30×10^5	1.64
	MAC	1	1.5/1b-op	

Functionality and SNR Validation: We verified functionality with software generated ground truth [9], [18], [20]. For volatile memory, we calibrated different types of SNR calculation with our internal SRAM-based IMC chip and Cadence simulations. For NVM-based IMC, we verify its SNR with our MRAM chip prototype, which is representative of the popular IMC of the NVM type. The average error in our SNR model is around 5% compared to chip measurements.

IV. Design Case Studies

In this section, we present case studies to highlight the use of IMCsim in designing IMC-based systems by demonstrating its ability to provide insights into performance, power, and area, of multiple designs as well as its ability to aid design verification of the prototype IC. We assume the generic architectural template in Fig. 4 is employed by the designer. The design configurations in Table III are employed for generating IMC designs. Three IMC types are considered: 1) SRAM AIMC; 2) MRAM AIMC; and 3) digital IMC (DIMC). Data for the IMC configurations were obtained from lab-tested/tape-out chip prototypes described in Section III.

We select three diverse workloads: 1) ResNet-18 [12] (11M parameters) on ImageNet [8] for the image classification task, representing CNN-type workloads; 2) Llama 6.7B [25] for the text generation task, representing transformer decoder-only autoregression workload; and 3) diffusion transformer [21], DiT-XL/2 (675M parameters) for the image generation task, representing transformer-based diffusion workloads [13]. For ResNet-18, the entire network is simulated and verified. However, without loss of generality, we simulate only one transformer layer of Llama and DiT, since the transformer model is built by repeating this layer.

A. Basic Profiling

Performance across IMC types: We employ the roofline plot [30] (in Fig. 5) to demonstrate how the three AI models perform on the three IMC processors, which are identical except for the IMC type.

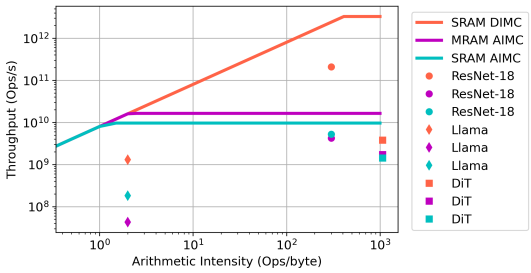


Fig. 5: The roofline plots generated by IMCsim.

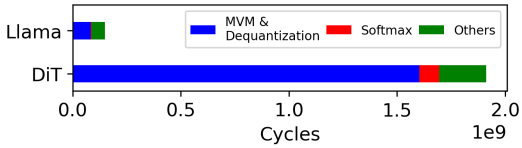


Fig. 6: Cycle-level breakdown for DIMC for sequence length = 256.

We find, DIMC achieves the highest peak throughput across all three workloads, which is consistent IMC benchmarking data [24]. This is due to DIMC's fast MAC operations, e.g., single-cycle MAC at 500 MHz) as compared to AIMC architectures, which rely on low ADC sampling rates, e.g., 20 MHz, to execute an MVM.

Performance across model types: ResNet-18 achieves the highest throughput due to high reuse enabled by its convolutional kernels and less frequent weight reloading. In Llama and DiT, MVM and dequantization dominate computation time when the sequence length is small, e.g., 256, which is consistent with the analysis in [22] as seen in Fig. 6. As a decoder-only transformer with a KV-cache, Llama performs MVM and softmax only on newly generated tokens. DiT, however, processes all tokens at each step, making its MVM and softmax operations much more computationally demanding.

B. Utilization Analysis

Utilization of IMC is measured by the number of memory rows and columns activated during MVM operations. Using DIMC as an example, Fig. 7 shows ResNet-18's utilization across layers. The first layer has the lowest utilization and longest run-time, while later layers sometimes reach 100% utilization due to larger kernels mapped to the IMC array. Periods of 0 utilization indicate idle IMC banks, possibly due to weight loading or waiting for other processes.

Utilization of transformer models is impacted by quantization group size. ResNet-18 uses per-layer quantization, but large models such as Llama and DiT require finer-grained quantization [10]. With group sizes of 64 for Llama and 128 for DiT, utilization fluctuates between 2 values - 0 and 12.5% or 25% - respectively, showing significant underutilization of the 512-row IMC bank and hence a lower throughput.

C. Few Large Cores vs. Many Small Cores

The choice between a few powerful cores and many small cores is critical in IMC design. Earlier AIMC designs [15] favored large cores, e.g., up to 1152 rows, but recent studies [26] have reduced core sizes to as few as 16 rows in DIMC. To investigate core size and number, we varied DIMC configurations for large models.

Table IV shows that splitting banks into smaller ones (*e.g.*, two 512×256 banks) offers no benefit, as columns are fully utilized in Llama and DiT. Scaling should instead focus on the row dimension. For example, though Llama is identified as memory-bound in the roofline plot (Fig. 5), its performance on DIMC can be improved

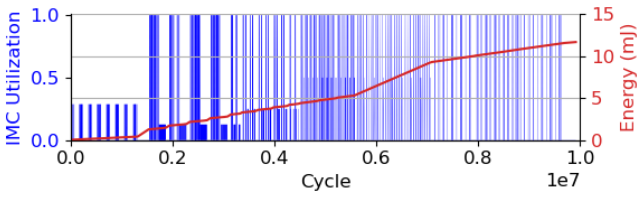


Fig. 7: Utilization and energy while executing ResNet-18 obtained via IMCsim's run-time profiling.

TABLE IV: Scaling results for large models obtained by IMCsim

Workload	DIMC bank specification	Number of banks	Relative peak throughput	Actual throughput	Average utilization (%)
Llama	512 × 256	2	1×	1.0×	0.09
	256 × 512	2	1×	2.0×	0.19
	128 × 512	2	0.5×	2.0×	0.19
	64 × 512	2	0.25×	2.0×	0.19
	64 × 512	4	0.5×	3.9×	0.36
DiT	512 × 256	2	1×	1.0×	1.9
	256 × 512	2	1×	2.0×	3.9
	128 × 512	2	0.5×	2.0×	3.9
	128 × 512	4	1×	3.9×	7.8
	128 × 512	8	2×	8.1×	15.4

by 2× by the use of improved bank dimensions (128 × 512) and more banks (2), which leads to enhanced utilization. Therefore, a key insight is that reducing IMC capacity can enhance throughput, especially when the number of IMC rows matches the quantization group size, thereby maximizing utilization. Another noteworthy observation is that the throughput enhancement in a multi-bank IMC can be slightly more than linear in the number of banks (8-bank IMC's throughput is 8.1× that of a single bank IMC) since multi-bank computations effectively mask idle cycles which single-bank IMCs are unable to.

These findings indicate multiple small cores are better suited for large models, paralleling trends in CPU design, where finer-grained scheduling and higher utilization are key for efficient hardware.

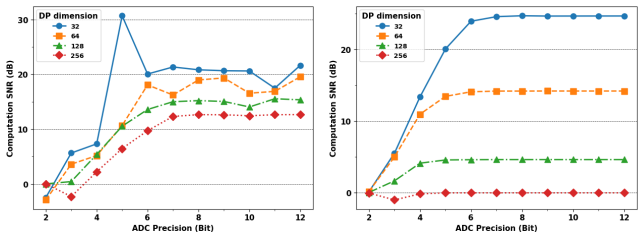
D. Compute SNR Analysis

Fig. 8 shows the impact of ADC bit precision on the computational SNR. It is generated by computing the DP of random input data. One observes that SRAM-based IMCs achieve higher SNR than MRAM-based ones. This is consistent with observed data [24]. The compute SNR for SRAM shows a peak when the ADC precision equals $\log_2(N)$, where N is the DP dimension. This peak occurs because each DP output has a unique ADC quantization level associated with it. Finally, the compute SNR increases as the N reduces, because the dynamic range of the ADC input is reduced. Such insights can guide circuit designers to choose ADC precision and dot product dimensions associated with specific quantization group sizes.

E. Energy Analysis

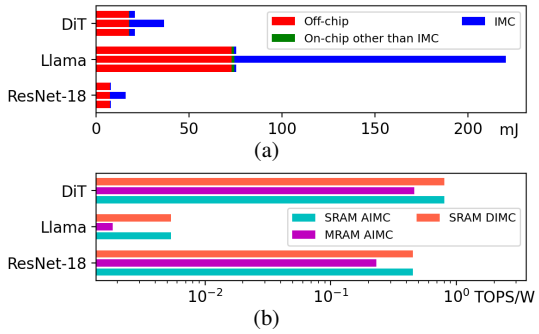
Fig. 9 profiles the system energy consumption and distribution for three AI workloads mapped to three IMC types. As expected, off-chip access dominates in all cases except MRAM-based IMC. The reason for this discrepancy is the high write energy of MRAM compared to SRAM and DIMC (see Table III). Furthermore, Llama requires more writes than DiT and ResNet-18 due to its much larger parameter count (6.7B vs. 675M for DiT and 11M for ResNet-18).

Interestingly, the energy efficiency of all applications is below 1 TOPS/W, which is about 10× lower than those reported in typical IC prototypes [24]. We conjecture this discrepancy results from accounting for the energy cost of frequent off-chip accesses which is typically not reported.



(a) (b)

Fig. 8: SNR for: (a) SRAM-, and (b) MRAM-based AIMCs.



(a)

(b)

Fig. 9: System energy profiling of three workloads mapped onto three IMC types (from top to bottom: DIMC, MRAM-based, and SRAM-based): (a) system energy distribution, and (b) system energy efficiency.

F. IMCsim as a Design Tool

This section illustrates the use of IMCsim as a design tool in implementing an prototype IMC chip.

Application specification: Consider the implementation of fixed-point MVMs kernels of one layer of a lightweight DiT algorithm on an IMC chip tailored for Edge having the following algorithmic parameters: *sequence length* = 256; *fixed-point precision* = 8-bit; *quantization group size* = 32 (attention) or 128 (otherwise); *MLP ratio* = 4. We assume non-MVM operations such as dequantization and non-linearity are executed off-chip.

Hardware specification: Assume an area budget of $4\,\mathrm{mm}^2$ in a $28\,\mathrm{nm}$ process. Based on the analyses in Section IV-B, the architecture needs to support a DP dimension consistent with the quantization group size (=32) to maximize utilization. Therefore, we choose a DIMC bank featuring 32 × 32 SRAM cells. The configuration file (see Fig. 1) will include the following parameters: chip area $A_{chip} = 4\,\mathrm{mm}^2$, percentage of area available for IMC and SRAM banks $P = 50\%$ (to leave room for the controller, pad ring and other support circuitry), standard SRAM bitcell with area $A_{bc} = 0.127\,\mu\mathrm{m}^2$; IMC bank unit area $A_{bank} = 16000\,\mu\mathrm{m}^2$; frequency $f = 500\,\mathrm{MHz}$; off-chip I/O bitwidth $B = 32$.

If the number of memory cells is M and the number of IMC banks is N_{bank}, then we have the following constraint:

$$\frac{A_{bc}}{F} M + A_{bank} N_{bank} \leq P A_{chip} \tag{4}$$

where F is the fraction of the total area of the SRAM banks that is occupied by bitcells.

Baseline Architecture: For the baseline case ($N_{bank} = 1$), with a typical value $F = 0.65$, (4) suggests a maximum on-chip memory capacity $M_{max} \leq 10^7$ (10 Mb). Employing the architectural template in Fig. 4 with one IMC 32 × 32 bank and 20 4096 × 128 SRAM banks, we employ IMCsim to profile the memory footprint of one

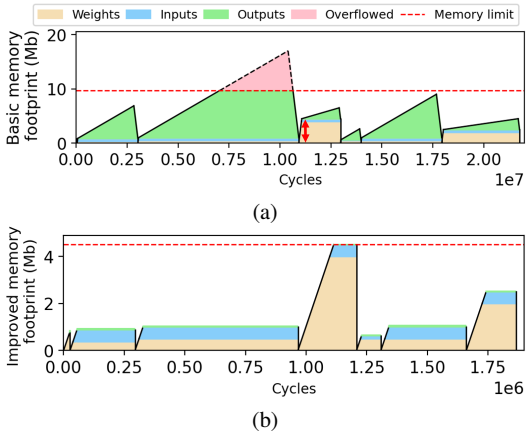


Fig. 10: On-chip memory footprint for one transformer block for: (a) the baseline architecture, and (b) the improved architecture.

transformer block as shown in Fig. 10(a). Note that overflow occurs during attention computation indicating that the memory management of the baseline architecture is not valid and needs to be improved.

Improved Architecture: To design an improved architecture, we employ IMCsim's run-time capability to determine what data (weights, inputs, and outputs) is immediately needed for MVM operations and which ones can be temporarily stored off-chip. The resulting reduction in memory requirements can be employed for additional IMC banks to enhance throughput.

First, we estimate the the maximum memory footprint for weights and inputs to be $4.48\,\text{Mb}$ (see red arrow at cycle no. 1.1×10^7 in Fig. 10(a)). Since each IMC bank generates a 128-bit (four 32-bit DPs) output, we get the smallest required memory footprint $M_{min} = 4.7 \times 10^6 + 128 \times N_{bank}$. Substituting M_{min} for M in (4) yields $N_{bank} \leq 67.2$. We choose $N_{bank} = 64$ to obtain $M_{min} = 4.49\,\text{Mb}$ which can be realized with 18 2048×128 SRAM banks.

The memory footprint (see Fig. 10(b)) of the improved architecture with 64 32×32 IMC banks and 18 2048×128 SRAM banks shows no memory overflow and a decrease in the overall latency by 90% due to the large number of IMC banks. Thus, this two-step analysis supported by IMCsim simulations is able to reveal a more effective architecture and memory management scheme.

Scheduling and Resource Allocation Analysis: IMCsim's cycle-level behavioral model enables detailed analysis of resource allocation and scheduling across time steps. The target is to identify and optimize underutilized resources. The resource allocation and scheduling of the improved architecture shown in Table V indicates that off-chip bus remains inactive after initial weights and inputs are loaded due to the single-port SRAM handling both off-chip and on-chip requests.

Table V also hints at the following optimizations to enhance utilization: 1) *memory optimization*: switching to dual-port SRAM can allow simultaneous off-chip reads and writes to reduce latency; partitioning SRAM into functional blocks (*e.g.*, input, weight, output SRAM) will help avoid SRAM bank conflicts; and 2) *computation optimization*: performing floating-point computations on-chip will reduce off-chip memory access.

Chip Implementation: We implemented the improved architecture in a $28\,\text{nm}$ process. We reused custom DIMC bank design from a recently completed chip design (to be reported in a separate publication). After verifying functionality in behavioral RTL with quantized software ground-truth, we synthesized the RTL using Genus, integrated the pre-designed DIMC bank layouts, and memory

TABLE V: Resource allocation and scheduling

Time Step	Off-chip Bus	SRAM	IMC Bank 0-63	ALU 0-63	Controller
1	Load	Write	Idle	Idle	Busy
2-24600	Load	Write	Idle	Idle	Idle
24601	Idle	Read	Write	Idle	Busy
24602-24632	Idle	Read	Write	Idle	Idle
24633	Idle	Read	Idle	Idle	Busy
24634-24645	Idle	Idle	MVM	Idle	Busy
24646	Idle	Write	Idle	Idle	Busy
24647	Idle	Read	Write	Idle	Busy
24648-24678	Idle	Read	Write	Idle	Idle
24679	Idle	Read	Idle	Idle	Busy
24680-24691	Idle	Idle	MVM	Idle	Busy
24692	Idle	Read	Idle	Idle	Busy
24693	Idle	Idle	Idle	Add	Busy
24694	Idle	Write	Idle	Idle	Busy
24695-24724	Idle	...	...	...	Busy
24725	Store	Read	Idle	Idle	Busy
24726-25044	Store	Read	Idle	Idle	Idle
25045-	...	...	...	...	...

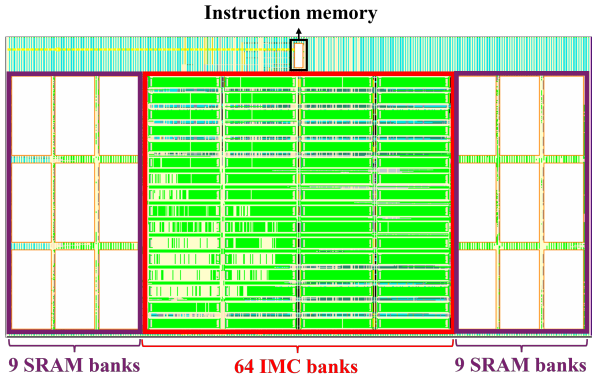


Fig. 11: A preliminary chip layout ($2.35\,\text{mm} \times 1.2\,\text{mm}$) in $28\,\text{nm}$ of an IMC processor for one transformer layer (pad ring omitted).

compiler-generated SRAM layouts. Next, we placed DIMC banks and SRAM banks (with a small instruction memory) and routed signals using Innovus, to generate a preliminary layout of the chip core implementing one transformer layer as shown in Fig. 11. This exercise demonstrates the use of IMCsim as a design tool.

V. Conclusion

This paper introduces IMCsim, a comprehensive, programmable, and extensible simulation framework and design tool for IMC systems. IMCsim is uniquely tailored to provide cycle-level simulation at the behavioral level, profiling run-time behavior, workload statistics, SNR, and other critical metrics necessary for designing IMC systems capable of executing large-scale deep learning workloads. To demonstrate its efficacy, we conducted detailed analyses and power, performance, area-aware design space evaluation which facilitate the IC design process. Our findings reveals new insights such as the impact of differing workload characteristics, the relation between quantization and utilization, the effect of off-chip access, and others. We demonstrated the capability and versatility of IMCsim, which positions it as a powerful tool for designing scalable IMC systems.

Acknowledgment

This work was supported by the Center for the Co-Design of Cognitive Systems (CoCoSys) funded by the Semiconductor Research Corporation (SRC) and the Defense Advanced Research Projects Agency (DARPA) under the JUMP 2.0 program.

REFERENCES

[1] T Andrulis, JS Emer, and V Sze. Cimloop: A flexible, accurate, and fast compute-in-memory modeling framework. In *2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2024.

[2] Aayush Ankit et al. Puma: A programmable ultra-efficient memristor-based accelerator for machine learning inference. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 715–731, 2019.

[3] Junjie Bai, Fang Lu, Ke Zhang, et al. Onnx: Open neural network exchange. https://github.com/onnx/onnx, 2019.

[4] Rajeev Balasubramonian et al. Cacti 7: New tools for interconnect exploration in innovative off-chip memories. *ACM Trans. Archit. Code Optim.*, 14(2), jun 2017.

[5] Fabrice Bellard. Qemu, a fast and portable dynamic translator. In *Proceedings of the Annual Conference on USENIX Annual Technical Conference*, ATEC '05, page 41, USA, 2005. USENIX Association.

[6] Hao Cai, Zhongjian Bian, Yaoru Hou, Yongliang Zhou, Jia-le Cui, Yanan Guo, Xiaoyun Tian, Bo Liu, Xin Si, Zhen Wang, Jun Yang, and Weiwei Shan. 33.4 a 28nm 2mb stt-mram computing-in-memory macro with a refined bit-cell and 22.4 - 41.5tops/w for ai inference. In *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, pages 500–502, 2023.

[7] Yu-Der Chih, Po-Hao Lee, Hidehiro Fujiwara, Yi-Chun Shih, Chia-Fu Lee, Rawan Naous, Yu-Lin Chen, Chieh-Pu Lo, Cheng-Han Lu, Haruki Mori, Wei-Chang Zhao, Dar Sun, Mahmut E. Sinangil, Yen-Huei Chen, Tan-Li Chou, Kerem Akarvardar, Hung-Jen Liao, Yih Wang, Meng-Fan Chang, and Tsung-Yung Jonathan Chang. 16.4 an 89tops/w and 16.3tops/mm2 all-digital sram-based full-precision compute-in memory macro in 22nm for machine-learning edge applications. In *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 64, pages 252–254, 2021.

[8] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.

[9] facebookresearch. facebookresearch/dit: Inference llama 2 in one file of pure c.

[10] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. OPTQ: Accurate quantization for generative pre-trained transformers. In *The Eleventh International Conference on Learning Representations*, 2023.

[11] An Guo, Xin Si, Xi Chen, Fangyuan Dong, Xingyu Pu, Dongqi Li, Yongliang Zhou, Lizheng Ren, Yeyang Xue, Xueshan Dong, Hui Gao, Yiran Zhang, Jingmin Zhang, Yuyao Kong, Tianzhu Xiong, Bo Wang, Hao Cai, Weiwei Shan, and Jun Yang. A 28nm 64-kb 31.6-tflops/w digital-domain floating-point-computing-unit and double-bit 6t-sram computing-in-memory macro for floating-point cnns. In *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, pages 128–130, 2023.

[12] Kaiming He, X. Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2015.

[13] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.

[14] Shubham Jain et al. Rxnn: A framework for evaluating deep neural networks on resistive crossbars. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 40(2):326–338, 2021.

[15] Hongyang Jia et al. 15.1 a programmable neural-network inference accelerator based on scalable in-memory computing. In *2021 IEEE International Solid- State Circuits Conference (ISSCC)*, volume 64, pages 236–238, 2021.

[16] Hongyang Jia, Hossein Valavi, Yinqi Tang, Jintao Zhang, and Naveen Verma. A programmable heterogeneous microprocessor based on bit-scalable in-memory computing. *IEEE Journal of Solid-State Circuits*, 55(9):2609–2621, 2020.

[17] Mingu Kang et al. An energy-efficient vlsi architecture for pattern recognition via deep embedding of computation in sram. In *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 8326–8330. IEEE, 2014.

[18] Karpathy. karpathy/llama2.c: Inference llama 2 in one file of pure c.

[19] Anni Lu et al. Neurosim simulator for compute-in-memory hardware accelerator: Validation and benchmark. *Frontiers in Artificial Intelligence*, 4, 2021.

[20] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019.

[21] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4195–4205, October 2023.

[22] Yubin Qin, Yang Wang, Dazheng Deng, Zhiren Zhao, Xiaolong Yang, Leibo Liu, Shaojun Wei, Yang Hu, and Shouyi Yin. Fact: Ffn-attention co-optimized transformer architecture with eager correlation prediction. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ISCA '23, New York, NY, USA, 2023. Association for Computing Machinery.

[23] Satyabrata Sarangi and Bevan Baas. Deepscaletool: A tool for the accurate estimation of technology scaling in the deep-submicron era. In *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5, 2021.

[24] Naresh R. Shanbhag and Saion K. Roy. Benchmarking in-memory computing architectures. *IEEE Open Journal of the Solid-State Circuits Society*, 2:288–300, 2022.

[25] Hugo Touvron et al. Llama: Open and efficient foundation language models, 2023.

[26] Fengbin Tu et al. A 28nm 15.59μj/token full-digital bitline-transpose cim-based sparse transformer accelerator with pipeline/parallel reconfigurable modes. In *2022 IEEE International Solid- State Circuits Conference (ISSCC)*, volume 65, pages 466–468, 2022.

[27] Kodai Ueyoshi, Ioannis A. Papistas, Pouya Houshmand, Giuseppe M. Sarda, Vikram Jain, Man Shi, Qilin Zheng, Sebastian Giraldo, Peter Vrancx, Jonas Doevenspeck, Debjyoti Bhattacharjee, Stefan Cosemans, Arindam Mallik, Peter Debacker, Diederik Verkest, and Marian Verhelst. Diana: An end-to-end energy-efficient digital and analog hybrid neural network soc. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 65, pages 1–3, 2022.

[28] Ashish Vaswani, Noam M. Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Neural Information Processing Systems*, 2017.

[29] Bo Wang, Chen Xue, Zhongyuan Feng, Zhaoyang Zhang, Han Liu, Lizheng Ren, Xiang Li, Anran Yin, Tianzhu Xiong, Yeyang Xue, Shengnan He, Yuyao Kong, Yongliang Zhou, An Guo, Xin Si, and Jun Yang. A 28nm horizontal-weight-shift and vertical-feature-shift-based separate-wl 6t-sram computation-in-memory unit-macro for edge depthwise neural-networks. In *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, pages 134–136, 2023.

[30] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM*, 52(4):65–76, apr 2009.

[31] Jinshan Yue, Chaojie He, Zi Wang, Zhaori Cong, Yifan He, Mufeng Zhou, Wenyu Sun, Xueqing Li, Chunmeng Dou, Feng Zhang, Huazhong Yang, Yongpan Liu, and Ming Liu. A 28nm 16.9-300tops/w computing-in-memory processor supporting floating-point nn inference/training with intensive-cim sparse-digital architecture. In *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, pages 1–3, 2023.

[32] Qilin Zheng et al. Pimulator-nn: An event-driven, cross-level simulation framework for processing-in-memory-based neural network accelerators. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(12):5464–5475, 2022.

[33] Zhenhua Zhu, Hanbo Sun, Tongxin Xie, Yu Zhu, Guohao Dai, Lixue Xia, Dimin Niu, Xiaoming Chen, Xiaobo Sharon Hu, Yu Cao, Yuan Xie, Huazhong Yang, and Yu Wang. Mnsim 2.0: A behavior-level modeling tool for processing-in-memory architectures. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(11):4112–4125, 2023.